GitHub Copilot no desenvolvimento de software: benefícios, limitações e revisão humana

José Kailon Teixeira da Silva
Carlos Henrique Nunes Ferreira Barbosa
Análise e Desenvolvimento de Sistemas | Design Profissional
Professora: Isabella Luiza dos Santos Souza
15 de outubro de 2026

Palavras-chave: GitHub Copilot; desenvolvimento de software; revisão de código.

Introdução

Receber uma sugestão de código não significa ter uma solução pronta. É preciso entender o que ela faz e verificar se atende ao problema. Essa questão orienta este trabalho sobre o GitHub Copilot, com foco no apoio à programação e nos limites de seu uso. A pergunta de pesquisa é: quais benefícios e dificuldades aparecem nos estudos selecionados e o que eles indicam sobre a revisão humana? O tema se relaciona à formação em Análise e Desenvolvimento de Sistemas, pois envolve decisões que o futuro profissional terá de justificar. O objetivo é analisar benefícios e limitações do Copilot na escrita e validação de código.

Metodologia

Foi realizada uma revisão bibliográfica narrativa, de natureza teórica e abordagem qualitativa. A busca exploratória ocorreu em 15 de outubro de 2026, na web, com os termos “GitHub Copilot”, “usability” e “security”, além dos títulos e nomes dos autores. Os textos completos foram consultados nos repositórios de Harvard, UC San Diego e arXiv, com conferência dos DOI das publicações ACM e IEEE. A seleção considerou estudos empíricos de 2021 a 2026, acesso integral e relação com a pergunta. Foram escolhidos três trabalhos publicados em 2022 e 2023. Materiais promocionais e versões duplicadas não compuseram o conjunto. Métodos, resultados e limitações foram comparados por tema, sem pretensão de esgotar a literatura.

Revisão da literatura

Usabilidade e apoio à programação

Vaithilingam, Zhang e Glassman (2022) compararam Copilot e IntelliSense com 24 participantes em tarefas Python. Embora 19 preferissem o Copilot, as diferenças no tempo de conclusão e no sucesso das tarefas não foram estatisticamente significativas. Os relatos também mostraram dificuldades para entender e corrigir o código sugerido. Barke, James e Polikarpova (2023), por sua vez, observaram 20 participantes em quatro linguagens e identificaram dois modos de uso: aceleração, quando a pessoa sabe como seguir, e exploração, quando procura uma solução. Os trabalhos apontam apoio para iniciar tarefas, mas medem aspectos diferentes. A descrição de um modo de aceleração não comprova, por si só, ganho geral de produtividade.

Segurança e validação do código

Pearce et al. (2022) examinaram 1.689 programas gerados em 89 cenários de segurança, com análise automatizada e inspeção manual. Cerca de 40% apresentaram vulnerabilidades. O resultado vale para os cenários escolhidos e não representa uma taxa de risco de todo uso do Copilot. Além disso, o conjunto inclui software e descrições de hardware. Enquanto os outros estudos observam pessoas usando a ferramenta, este avalia as saídas geradas. A diferença impede somar os resultados como se fossem uma única medida. Para esta revisão, o diálogo entre eles reforça que facilidade de uso e segurança precisam ser avaliadas separadamente.

Síntese crítica

O ponto central da comparação é o trabalho que continua depois da geração do código. Uma sugestão pode ajudar a começar uma tarefa e, ainda assim, exigir correções. Também pode parecer adequada sem ter sido examinada quanto à segurança. Por isso, a utilidade deve ser discutida junto ao esforço de revisão. Há limites claros nas evidências: amostras pequenas de usuários, tarefas delimitadas e versões antigas da ferramenta. Nesse conjunto, falta acompanhar equipes por mais tempo para relacionar o apoio recebido ao retrabalho e aos defeitos encontrados nos projetos.

Considerações finais

Os estudos indicam apoio à escrita de código, mas também dificuldades de compreensão e riscos que não permitem dispensar a revisão humana. A resposta à pergunta depende, portanto, da tarefa e do modo como a sugestão é verificada. Na formação profissional, propõe-se que o aluno explique o código aceito e demonstre, por meio de testes, como ele atende aos requisitos. Essa proposta é uma interpretação da revisão, não uma intervenção avaliada. Pesquisas futuras podem comparar o esforço de validação em projetos completos. As conclusões não devem ser transferidas automaticamente às versões atuais do Copilot.

Resumo

Esta revisão narrativa compara três estudos sobre o GitHub Copilot, publicados em 2022 e 2023. A análise qualitativa aborda usabilidade e segurança. Os trabalhos mostram apoio à programação, dificuldades de compreensão e limites no código gerado. Conclui-se que o uso do assistente exige revisão humana e verificação técnica, respeitados os limites das pesquisas analisadas.

Referências

BARKE, Shraddha; JAMES, Michael B.; POLIKARPOVA, Nadia. Grounded Copilot: How Programmers Interact with Code-Generating Models. Proceedings of the ACM on Programming Languages, v. 7, n. OOPSLA1, art. 78, 27 p., 2023. DOI: https://doi.org/10.1145/3586030.

PEARCE, Hammond; AHMAD, Baleegh; TAN, Benjamin; DOLAN-GAVITT, Brendan; KARRI, Ramesh. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In: 2022 IEEE Symposium on Security and Privacy (SP). IEEE, 2022. p. 754-768. DOI: https://doi.org/10.1109/SP46214.2022.9833571.

VAITHILINGAM, Priyan; ZHANG, Tianyi; GLASSMAN, Elena. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In: CHI Conference on Human Factors in Computing Systems Extended Abstracts. New York: ACM, 2022. 7 p. DOI: https://doi.org/10.1145/3491101.3519665.